

Ph. D. Juan David Martínez Vargas
Ph. D. Raul Andrés Castañeda

Escuela de Ciencias Aplicadas e Ingeniería

Lecture 05

Deep Learning

Agenda

- Motivación y MNIST
- Pipeline end-to-end
- Dataset vs DataLoader
- Tensores, transforms y normalización
- Modelo fully connected
- Loss, optimizer y accuracy
- Training / validation + early stopping
- Curvas, inferencia y matriz de confusión
- Guardar y cargar el modelo
- Quiz + puente hacia CNNs

El "Hello word" del DL

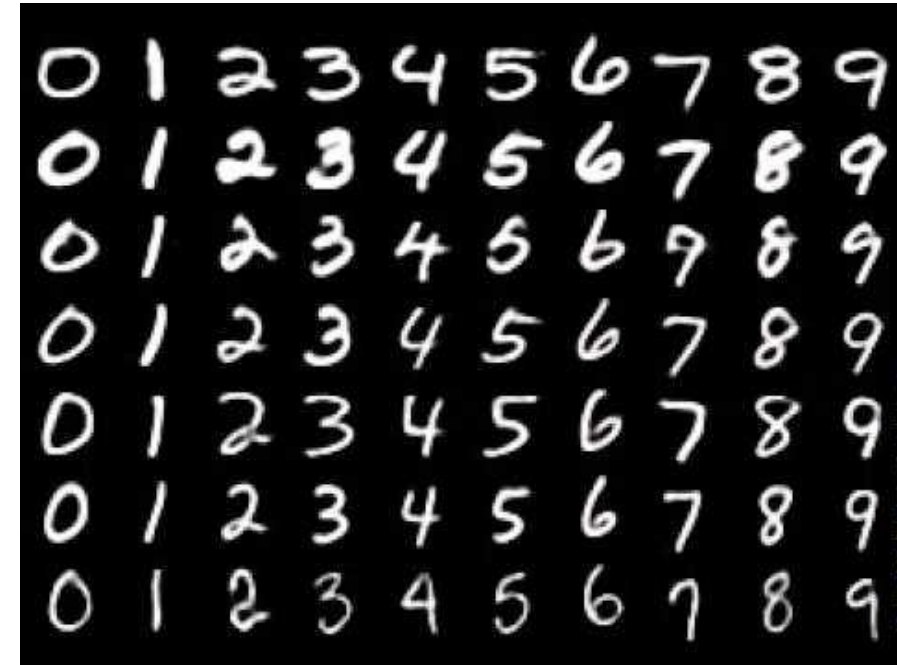
Clasificación de dígitos manuscritos usando PyTorch

En este Ejemplo se implementará un modelo de clasificación supervisada para reconocer dígitos manuscritos (0–9) a partir de imágenes en escala de grises, utilizando la base de datos **MNIST** y la librería PyTorch.

El objetivo es entrenar un modelo que, dada una imagen de entrada, asigne correctamente la probabilidad de pertenencia a cada una de las 10 clases posibles.

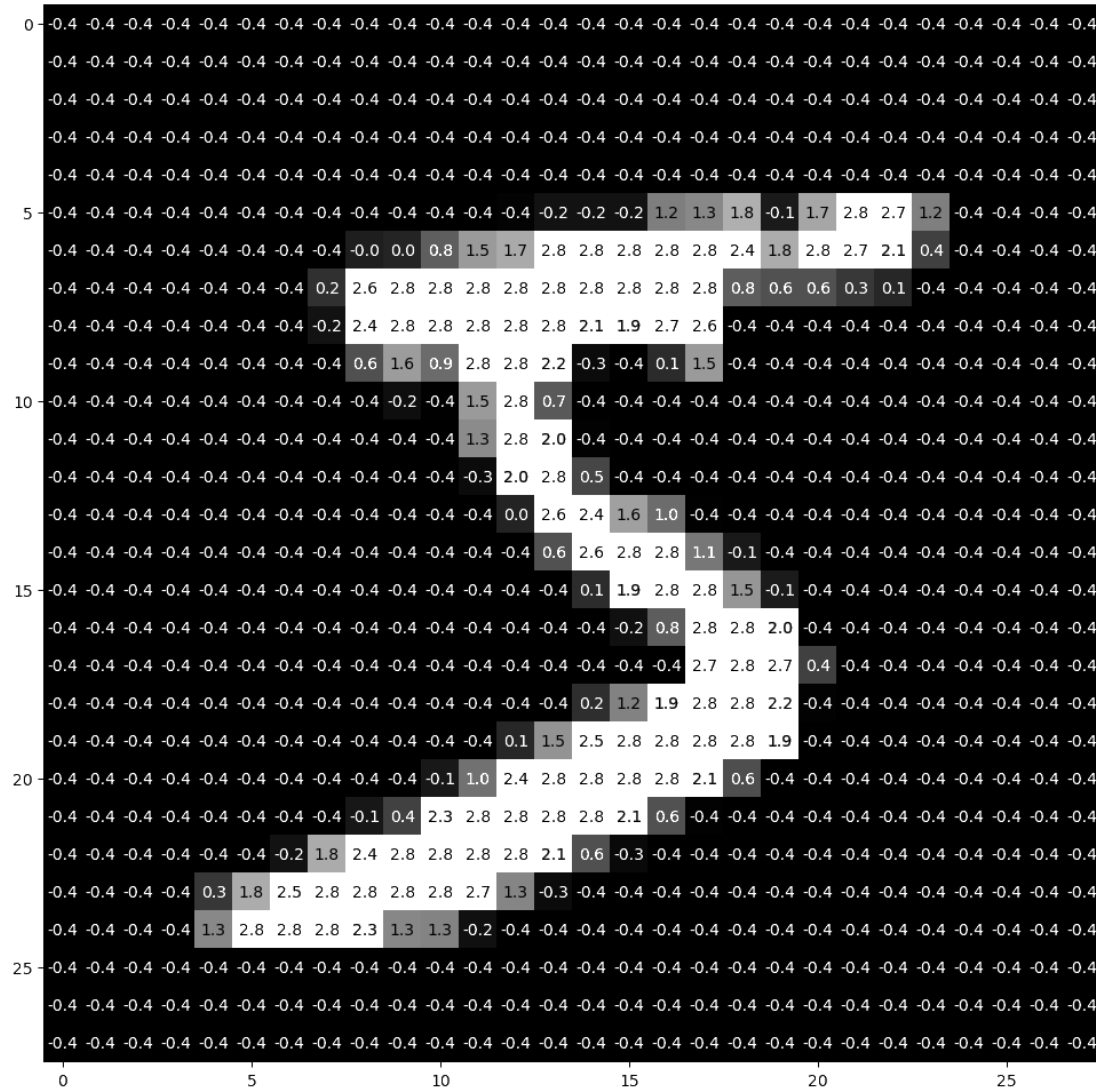
El ejercicio se desarrollará paso a paso, abordando los siguientes aspectos:

- Carga y exploración del conjunto de datos MNIST.
- Preparación de los datos para el entrenamiento (transformaciones y normalización).
- Definición de un modelo de clasificación basado en capas totalmente conectadas (Fullyconnected).
- Implementación de la función de pérdida y del algoritmo de optimización.
- Entrenamiento del modelo y análisis de la evolución de la pérdida.
- Evaluación del desempeño del modelo sobre datos no vistos.



El "Hello word" del DL

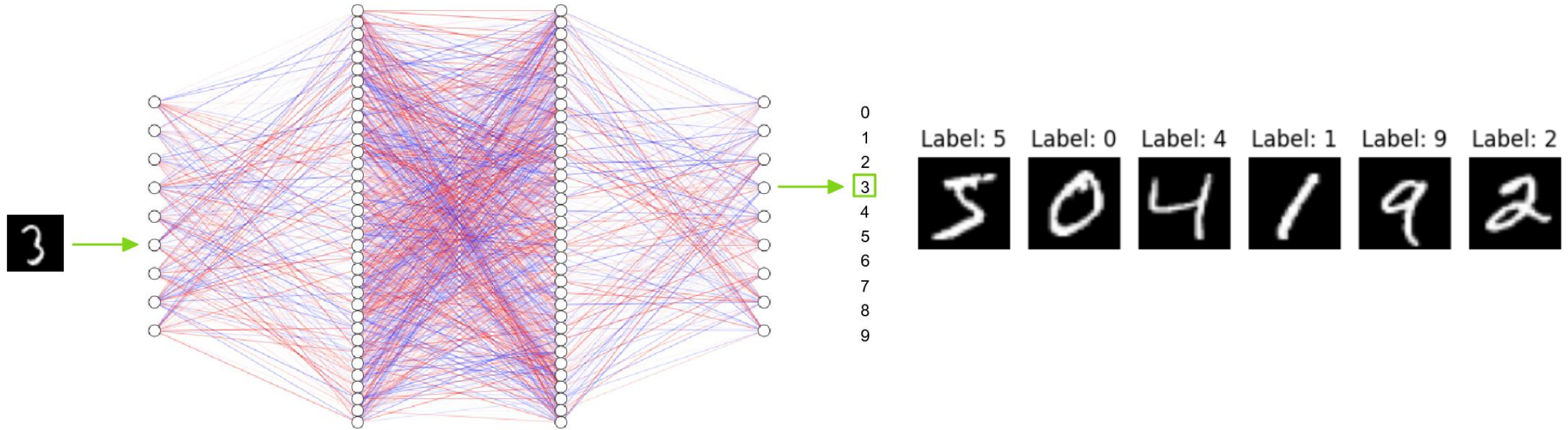
¿ Qué es MNIST ?



- Contiene **60.000** imágenes de entrenamiento y **10.000** imágenes de prueba.
- Cada imagen tiene un tamaño de 28×28 píxeles y representa un dígito escrito a mano.
- 10 clases: dígitos 0–9
- Las imágenes están en escala de grises.
- Cada imagen está asociada a una etiqueta.



El "Hello word" del DL



Pregunta: ¿qué reglas escribirías para reconocer todos los posibles "7"?

El "Hello word" del DL

Pipeline para implementar un modelo DL end-to-end

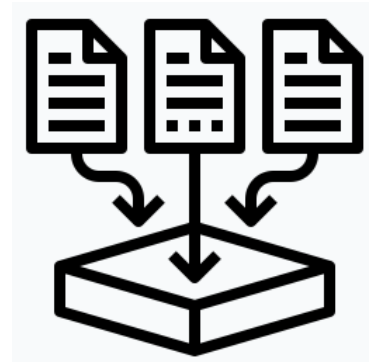


- Separar claramente datos, modelo, aprendizaje y evaluación.
- Este patrón se repite en proyectos mucho más grandes.

El "Hello word" del DL

Para implementar un modelo DL end-to-end

- Cargar y explorar la base de datos (ver ejemplos, tamaños, clases).
- Preprocesar (transformaciones, normalización, flatten si aplica).
- Definir conjuntos de entrenamiento y validación/prueba + DataLoader.
- Definir la arquitectura de la red neuronal (capas, activaciones).
- Configurar el aprendizaje: función de pérdida, optimizador, métricas (accuracy).
- Entrenar el modelo (épocas, monitoreo de loss/accuracy).
- Evaluar y discutir resultados (curvas, errores típicos, overfitting) (opcional: guardar modelo).



**Recollect
data**



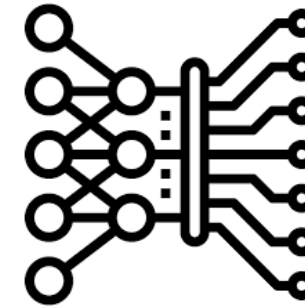
**Data pre-
processing**



Data Analysis



Split data



Model



Evaluation

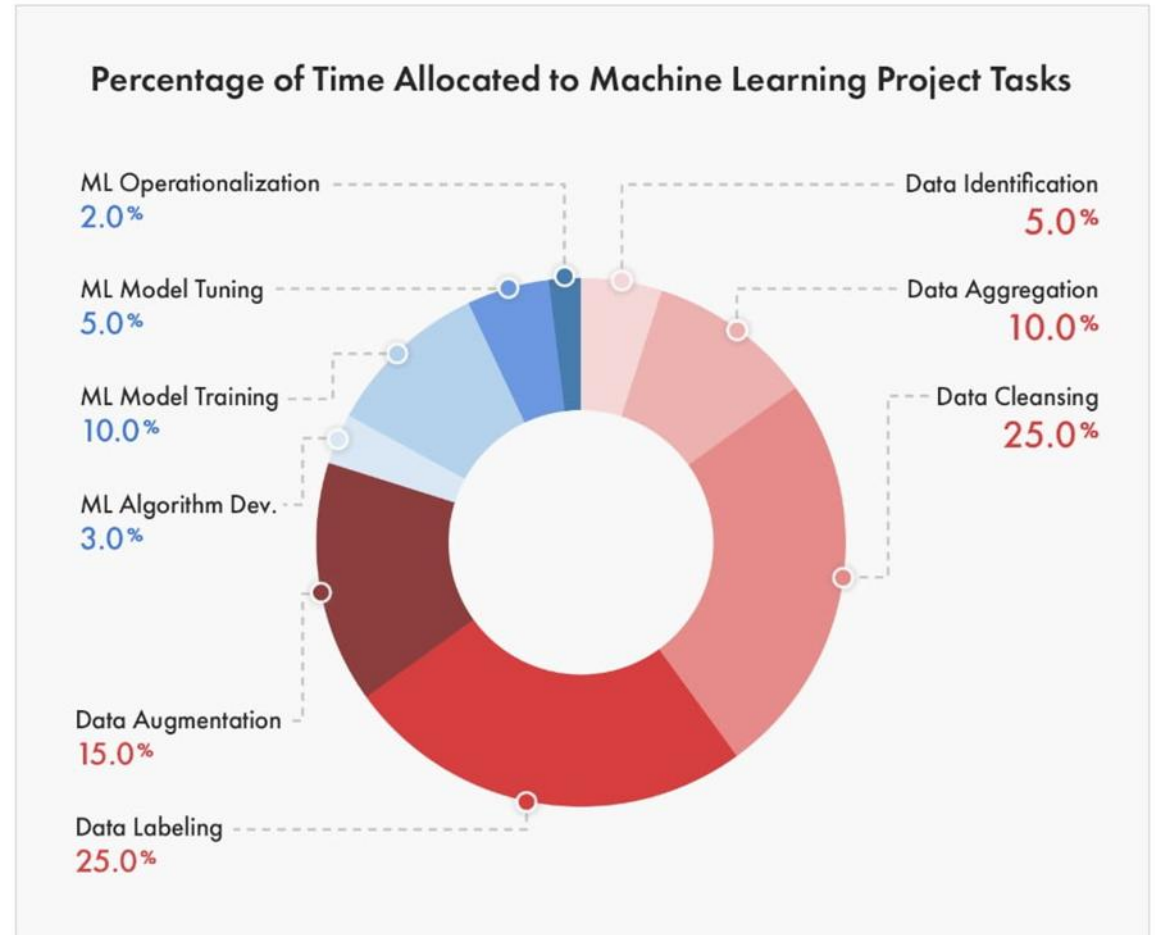
Data Splitting



<https://builtin.com/data-science/train-test-split>

<https://kili-technology.com/blog/training-validation-and-test-sets-how-to-split-machine-learning-data>

El "Hello word" del DL



[Link](#)

El "Hello word" del DL

Train

- Aprende los parámetros del modelo
- Forward + backpropagation
- Normalmente shuffle=True

Validation / Test

- No actualiza pesos
- Mide generalización
- Normalmente shuffle=False

Train = estudiar ·
Validation = practicar el examen
Test = examen final



El "Hello word" del DL

Step-by-step

```
!pip install torch torchvision torchaudio
```

4m 12.2s

Python

**Instalar
Librerías**

```
!pip install matplotlib
```

4.7s

Python

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torch.optim import Adam

# Visualization tools
import torchvision
import torchvision.transforms.v2 as transforms
import torchvision.transforms.functional as F
import matplotlib.pyplot as plt
```

**Importar
Librerías**

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.cuda.is_available()
```

False

Verificar GPU disponible

Step-by-step Dataset vs DataLoader

Dataset = ¿qué datos existen?

- Define cómo acceder a una muestra
- Implementa `__len__`
- Implementa `__getitem__`
- Devuelve típicamente (x, y)

DataLoader = ¿cómo se entregan?

- Crea batches
- Puede mezclar con shuffle
- Puede cargar en paralelo
- Itera durante entrenamiento

[Link](#)

```
import torch
from torch.utils.data import Dataset
from torchvision import datasets
from torchvision.transforms import v2
import matplotlib.pyplot as plt
```

```
training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)])
)

test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)])
)
```

Step-by-step Dataset vs DataLoader

```
import os
import pandas as pd
from torchvision.io import decode_image

class CustomImageDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform=None, target_transform=None):
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.target_transform = target_transform

    def __len__(self):
        return len(self.img_labels)

    def __getitem__(self, idx):
        img_path = os.path.join(self.img_dir, self.img_labels.iloc[idx, 0])
        image = decode_image(img_path)
        label = self.img_labels.iloc[idx, 1]
        if self.transform:
            image = self.transform(image)
        if self.target_transform:
            label = self.target_transform(label)
        return image, label
```

La clase CustomImageDataset hereda de la clase Dataset

Step-by-step Dataset vs DataLoader

```
def __init__(self, images, labels, transform=None):
    self.images = images          # e.g. a NumPy array of shape (N, 28, 28)
    self.labels = labels          # e.g. a NumPy array of shape (N,)
    self.transform = transform    # optional preprocessing (see Section 7)

def __len__(self):
    # DataLoader calls this to know how many samples exist
    return len(self.images)

def __getitem__(self, idx):
    # DataLoader calls this (with a random or sequential idx) to fetch one sample
    image = self.images[idx]
    label = int(self.labels[idx])
    if self.transform:
        image = self.transform(image)
    return image, label
```

`_init_` → **Inicializa el Dataset.** Recibe o carga los datos y almacena la información necesaria, por ejemplo, las rutas de las imágenes y sus labels desde un archivo CSV.

`_len_` → **Devuelve el número total de muestras** disponibles en el Dataset.

`_getitem_` → **Devuelve una muestra específica** (image, label) de acuerdo con el índice idx

El "Hello word" del DL

Step-by-step Dataset vs DataLoader

DataLoader → Se encarga de entregar la data al modelo.

Es una clase iteradora.

Los ciclos se encargan de cargar la data mediante el *batch_size*.

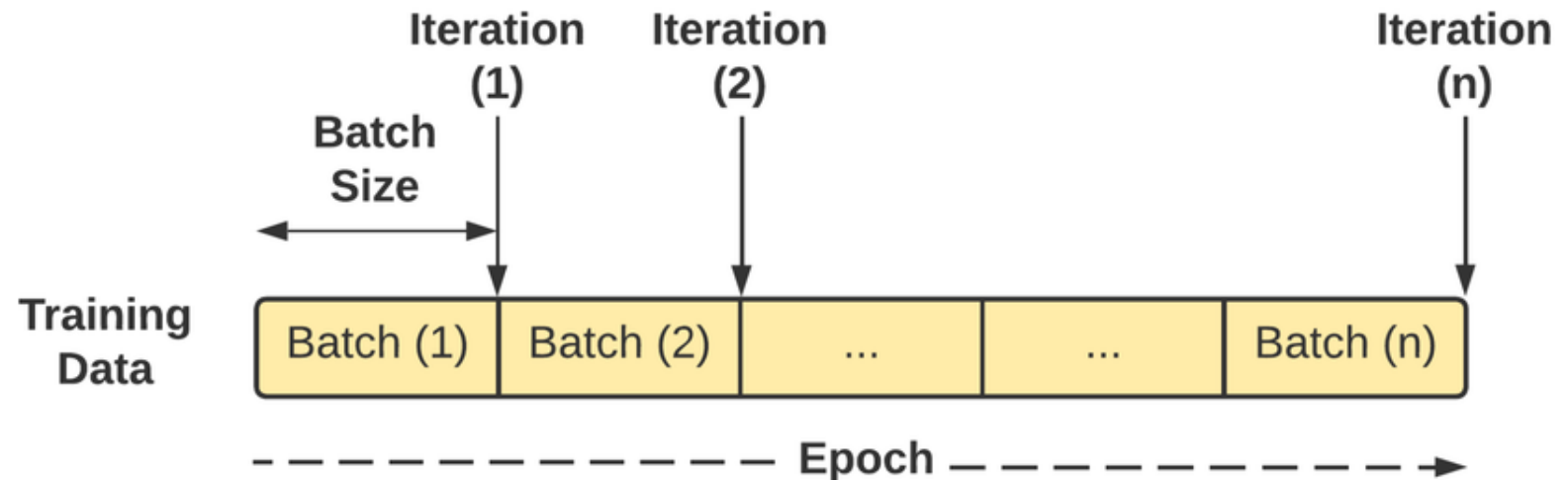
```
batch_size = 32

train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
valid_loader = DataLoader(valid_set, batch_size=batch_size)
```

El modelo no aprende con todos los datos a la vez, aprende por lotes

Entre los hiperparámetros, el **learning rate** y el **batch size** son dos parámetros directamente relacionados con el algoritmo del **gradient descent**.

El **batchsize** se refiere al número de muestras utilizadas en cada uno de estos lotes durante el entrenamiento.



Step-by-step Dataset vs DataLoader

How does Batch Size impact your model learning

Different aspects that you care about

[Link](#)

Batch 1

Batch 2

Batch 3

Batch 4

Batch 5

Batch 6

Batch 1 → forward → loss → backward → update
Batch 2 → forward → loss → backward → update
...
Batch 1875 → forward → loss → backward → update

1 batch = una iteración de entrenamiento.
1 epoch = todos los batches necesarios para recorrer las 60,000 muestras una vez.

El "Hello word" del DL

Step-by-step Dataset vs DataLoader

Batch 1 → forward → loss → backward → update
Batch 2 → forward → loss → backward → update
...
Batch 1875 → forward → loss → backward → update

1 batch = una iteración de entrenamiento.
1 epoch = todos los batches necesarios para recorrer las 60,000 muestras una vez.

EPOCH 1

TRAINING

Batch 1 → Forward → Loss → Backward → Update weights
Batch 2 → Forward → Loss → Backward → Update weights
...
Batch n → Forward → Loss → Backward → Update weights

AQUÍ TERMINÓ LA ÉPOCA DE ENTRENAMIENTO

VALIDATION

Batch 1 → Forward → Loss / Accuracy
Batch 2 → Forward → Loss / Accuracy
...

✗ No Backward
✗ No weight update

El "Hello word" del DL

Step-by-step

```
train_set = torchvision.datasets.MNIST("./data/", train=True, download=True)  
valid_set = torchvision.datasets.MNIST("./data/", train=False, download=True)
```

Python

Dividir los
datos

Dataset MNIST

Number of datapoints: 60000

Root location: ./data/

Split: Train

Dataset MNIST

Number of datapoints: 10000

Root location: ./data/

Split: Test

```
x_0, y_0 = train_set[0]
```

```
x_0
```

```
type(x_0)
```

```
y_0
```

```
type(y_0)
```

5

int



PIL.Image.Image

El "Hello word" del DL

Step-by-step

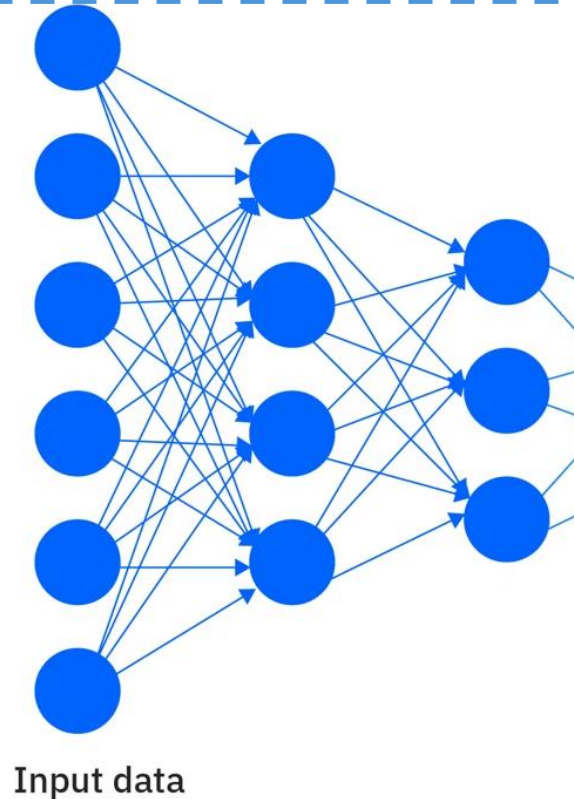
```
trans = transforms.Compose([transforms.ToTensor()])  
x_0_tensor = trans(x_0)
```

Convertir datos a
tensores

El resultado **'x_0_tensor'** es: imagen convertida a tensor

¿Por qué es importante?* PyTorch solo trabaja con tensores → Es como traducir de español a inglés para que PyTorch "entienda" los datos.

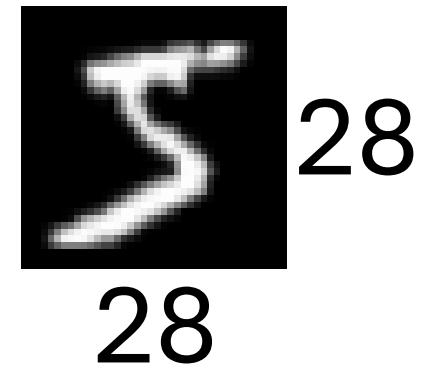
Además, normaliza los valores: convierte los píxeles de rango [0, 255] a rango [0.0, 1.0].



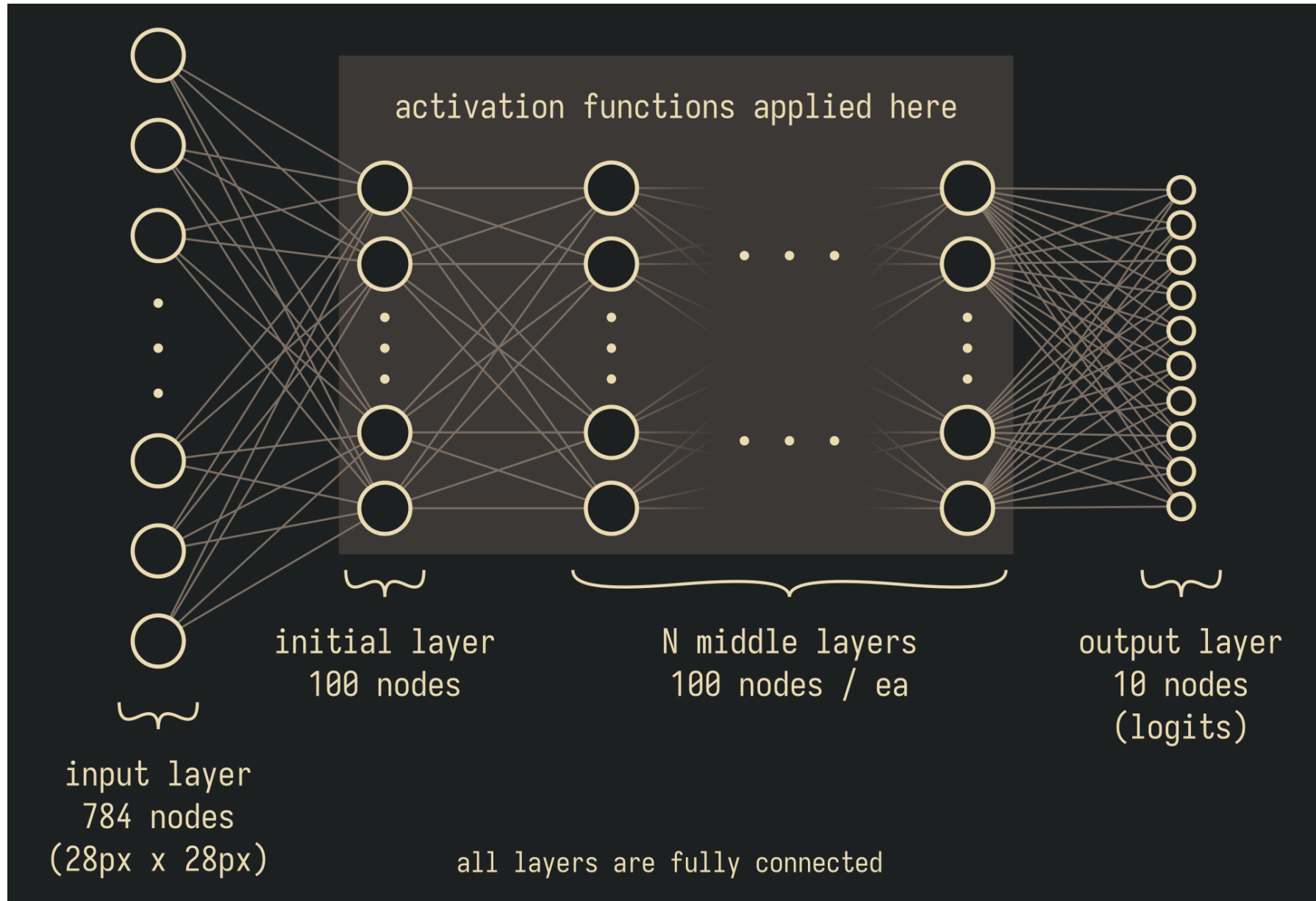
El modelo no "ve" imágenes, ve
Tensores.

$$x \in R^{784}$$

```
x_0_tensor.size()  
  
torch.Size([1, 28, 28])
```



El "Hello word" del DL

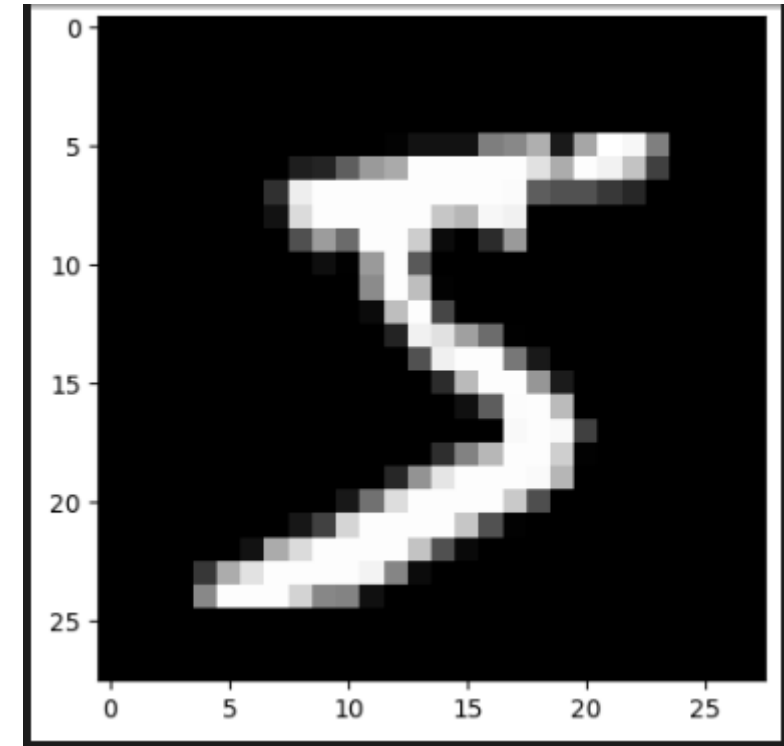


El "Hello word" del DL

Step-by-step

```
tensor([[[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0118, 0.0706, 0.0706, 0.0706,
          0.4941, 0.5333, 0.6863, 0.1020, 0.6510, 1.0000, 0.9686, 0.4980,
          0.0000, 0.0000, 0.0000, 0.0000],
         [0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
          0.0000, 0.0000, 0.0000, 0.0000],
         ...])
```

```
image = F.to_pil_image(x_0_tensor)
plt.imshow(image, cmap='gray')
```



**Datos
normalizados**

El "Hello word" del DL

Step-by-step

```
batch_size = 32

train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
valid_loader = DataLoader(valid_set, batch_size=batch_size)
```

Balance entre velocidad, estabilidad y generalización

¿Por qué mezclar (`shuffle`)?

• En **entrenamiento**:

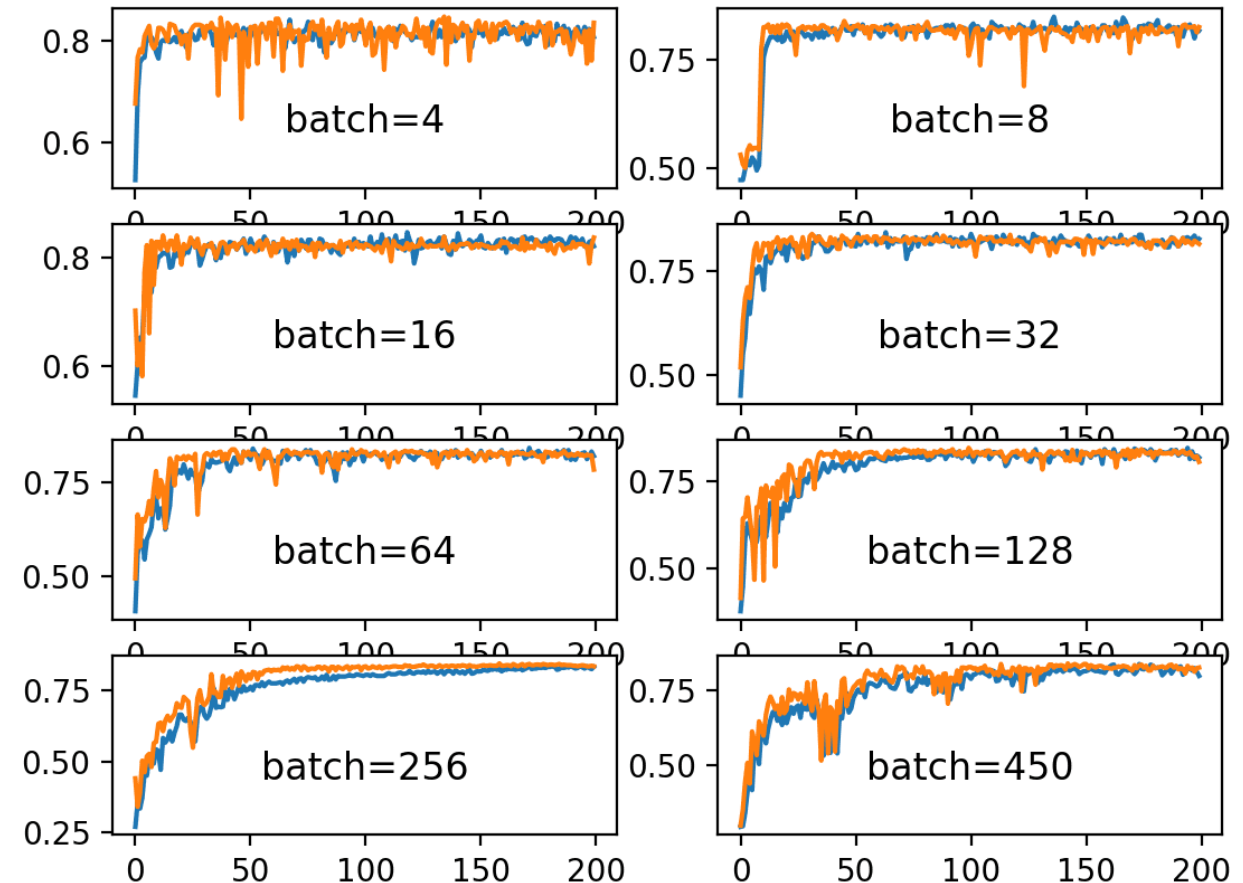
- Se mezcla para evitar que el modelo aprenda patrones por orden

• En **validación**:

- No es necesario mezclar

¿Por qué no usar todo el dataset de una vez?

- Consume muchos recursos (RAM / GPU)
- Es menos eficiente
- Puede hacer el aprendizaje inestable
- El gradiente sería muy costoso de calcular

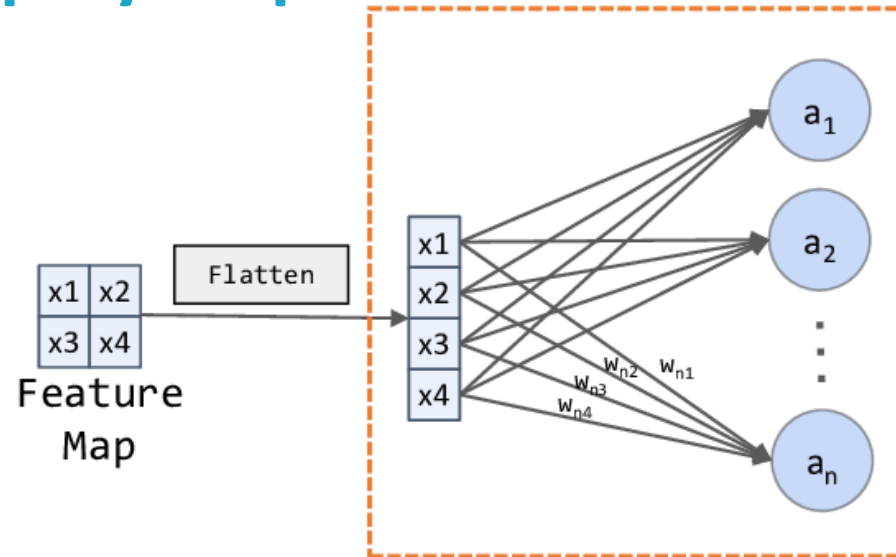


[Link](#)

El "Hello word" del DL

Step-by-step

Fully Connect Networks



Convierte una imagen multidimensional en un vector de una sola dimensión.

1	1	0
4	2	1
0	2	1

`nn.Flatten()`

1
1
0
4
2
1
0
2
1

```
layers = [  
    nn.Flatten()  
]  
layers
```

```
[Flatten(start_dim=1, end_dim=-1)]
```

El "Hello word" del DL

Step-by-step

Convierte una imagen multidimensional en un vector de una sola dimensión.

```
layers = [  
    ... nn.Flatten()  
]  
layers
```

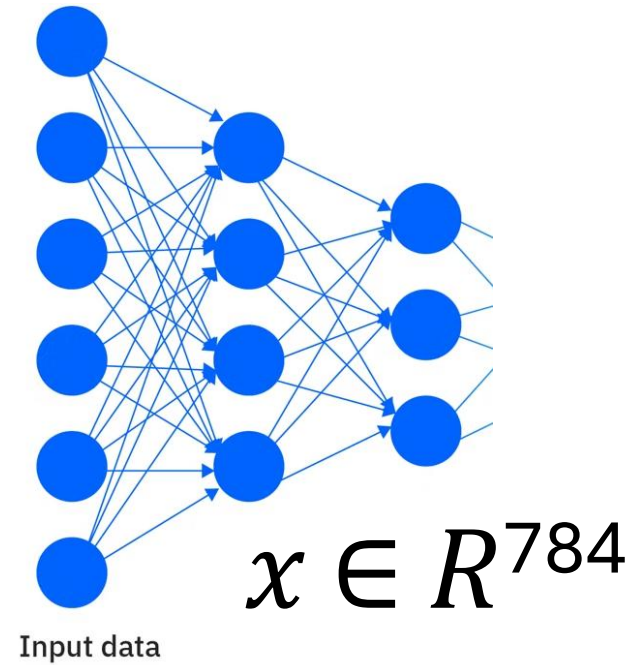
```
[Flatten(start_dim=1, end_dim=-1)]
```

Imagen MNIST original: Forma: (1, 28, 28) → 1 canal, 28x28 píxeles

```
[  
    [pixel1, pixel2, ..., pixel28],  
    [pixel29, pixel30, ..., pixel56],  
    ...  
    [pixel757, pixel758, ..., pixel784]]  
]
```

Después de Flatten: Forma: (784,) → un vector largo

```
[  
    pixel1, pixel2, pixel3, ..., pixel783, pixel784  
]
```



El "Hello word" del DL

Step-by-step

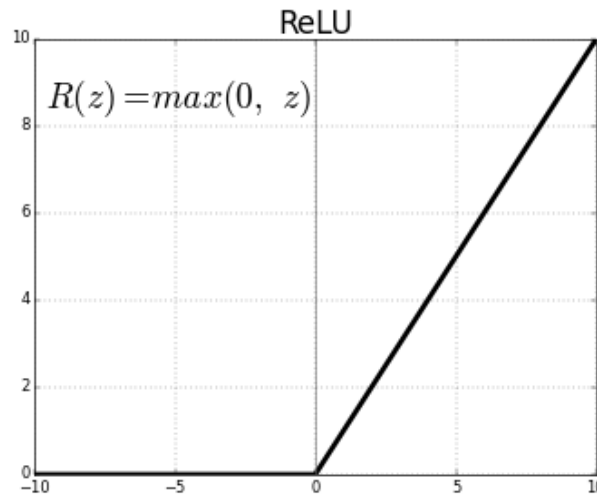
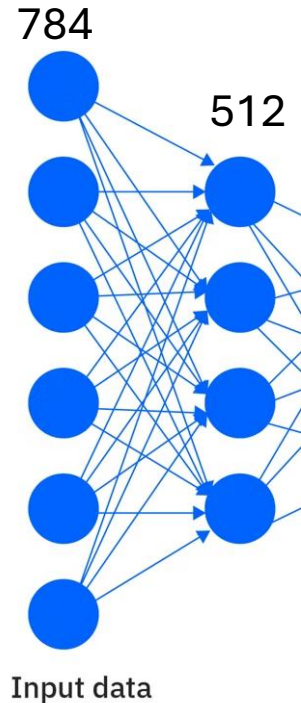
```
layers = [  
    nn.Flatten(),  
    nn.Linear(input_size, 512), # Input  
    nn.ReLU(), # Activation for input  
]  
layers
```

Entradas y primera capa

“

For example, the rectified linear function $g(z) = \max\{0, z\}$ is not differentiable at $z = 0$. This may seem like it invalidates g for use with a gradient-based learning algorithm. In practice, gradient descent still performs well enough for these models to be used for machine learning tasks.

— Page 192, [Deep Learning](#), 2016



Antes de ReLU: [-2.3, 0.5, -1.1, 3.2]
Después de ReLU: [0, 0.5, 0, 3.2]

Try playing around with this value later to see how it affects training and to start developing a sense for what this number means

El "Hello word" del DL

Step-by-step

```
n_classes = 10

layers = [
    nn.Flatten(),
    nn.Linear(input_size, 512), # Input
    nn.ReLU(), # Activation for input
    nn.Linear(512, 512), # Hidden
    nn.ReLU(), # Activation for hidden
    nn.Linear(512, n_classes) # Output
]
layers
```

Agregar capas
ocultas y de
salida

parametros_gpt3 = 175.000.000.000

memoria_gpt3_gb = (parametros_gpt3 * 4) / (1024**3)

Resultado: 650 GB ✗ ¡No cabe en tu GPU de 8 GB

parámetros = (input * output) + bias

➤ Capa 1: Linear(784, 512)

$$(784 * 512) + 512 = 401.920$$

➤ Capa 2: Linear(512, 512)

$$(512 * 512) + 512 = 262.656$$

➤ Capa 3: Linear(512, 10)

$$(512 * 10) + 10 = 5.130$$

¿Por qué es importante saber el número de parámetros?

Más parámetros = más RAM/GPU necesaria- 669,706

parámetros × 4 bytes (float32) ≈ **2.7 MB**

Pocos parámetros → Underfitting (no aprende bien)

Muchos parámetros → Overfitting (memoriza, no generaliza)

El "Hello word" del DL

Step-by-step Construcción del modelo fully connected

Sequential → es la forma más simple de construir una red neuronal en PyTorch.

```
model = nn.Sequential(*layers)
model
```

⊗ 0.2s

Modelo secuencial

```
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 10)
).to(device)
```



? Sin asterisco

`nn.Sequential(layers)`

Pasa UNA lista

✓ Con asterisco

`nn.Sequential(*layers)`

Desempaqueta → argumentos individuales

Analogía: Desempaquetar una caja

`lista = ['a', 'b', 'c']`

`funcion(lista)` → pasa la caja completa

`funcion(*lista)` → pasa 'a', 'b', 'c' por separado

[Link](#)

El "Hello word" del DL

Step-by-step

```
model = torch.compile(model)
```

**Optimización
del modelo**

**Definir la función
de pérdida**

```
loss_function = nn.CrossEntropyLoss()
```

CrossEntropy: está diseñado para calificar si un modelo predijo la categoría correcta de un grupo de categorías.

$$CE = - \sum_{i=1}^N y_i \log(p_i)$$

[Link](#)

Clase:	0	1	2	3	4	5	6	7	8	9
Logit:	0.2	-1.3	0.8	1.1	-0.5	0.4	3.7	0.1	-0.8	0.5
							↑			
							mayor valor			

```
prediction = logits.argmax(dim=1)
```

Ventajas:

Más rápido - Especialmente en GPUs modernas.

Fácil de usar - Solo una línea de código.

Compatible - Funciona con la mayoría de modelos.

Sin cambiar código - Tu modelo sigue funcionando igual.

Desventajas:

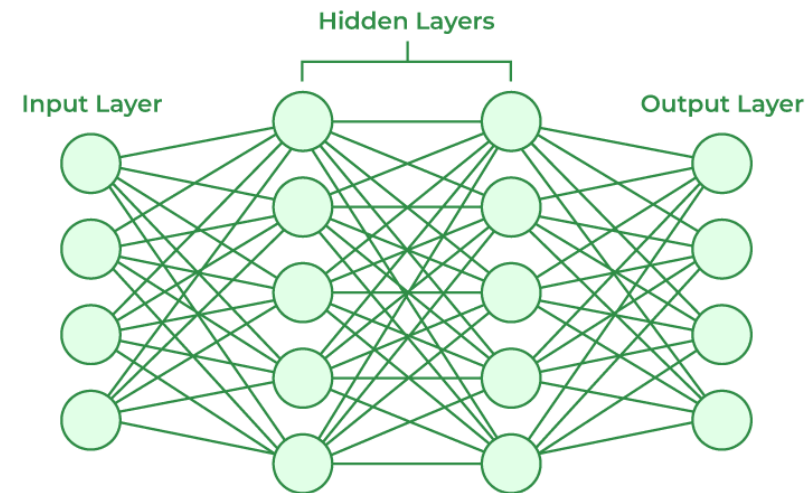
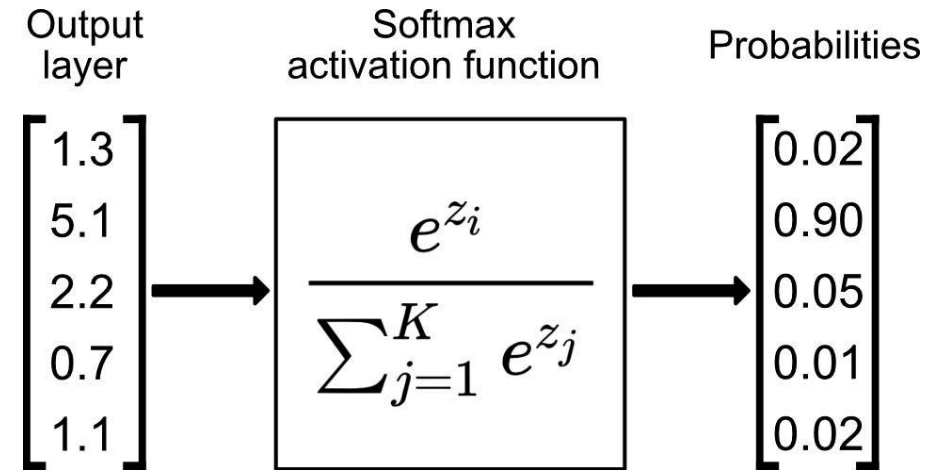
No siempre más rápido - En modelos muy pequeños puede ser peor.

El "Hello word" del DL

Regresión *softmax*

$$\text{softmax}(e^{z_i}) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Queremos una sola clase ganadora
Queremos probabilidades interpretables
Funciona perfectamente con Cross-Entropy
Loss



El "Hello word" del DL

Step-by-step

Definir la función de pérdida

```
loss_function = nn.CrossEntropyLoss()
```

Clase:	0	1	2	3	4	5	6	7	8	9
Logit:	0.2	-1.3	0.8	1.1	-0.5	0.4	3.7	0.1	-0.8	0.5

↑
mayor valor

Importante: durante el entrenamiento con `nn.CrossEntropyLoss()` le entregamos directamente los **logits**, no necesitamos aplicar `Softmax` antes, porque `CrossEntropyLoss` hace internamente la transformación necesaria.

CrossEntropy: está diseñado para calificar si un modelo predijo la categoría correcta de un grupo de categorías. [Link](#)

$$CE = - \sum_{i=1}^N y_i \log(p_i)$$

LOGITS

PROBABILITIES

0 → 0.2

0 → 0.02

1 → -1.3

1 → 0.01

2 → 0.8

2 → 0.04

...

...

6 → 3.7

6 → 0.78

...

...

Sum = 1.00

Softmax

El "Hello word" del DL

Step-by-step

$$CE = - \sum_{i=1}^N y_i \log(p_i) \quad \text{softmax}(e^{z_i}) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Supongamos → Etiqueta verdadera: $y = 6$

Softmax produce:

Class	Logit	Probability
0	0.2	0.023
1	-1.3	0.005
2	0.8	0.042
3	1.1	0.057
4	-0.5	0.012
5	0.4	0.028
6	3.7	0.771
7	0.1	0.021
8	-0.8	0.009
9	0.5	0.031



[1,0,0,0,0,0,0,0,0,0]



[0,1,0,0,0,0,0,0,0,0]



[0,0,1,0,0,0,0,0,0,0]



[0,0,0,1,0,0,0,0,0,0]



[0,0,0,0,1,0,0,0,0,0]



[0,0,0,0,0,0,0,0,1,0]



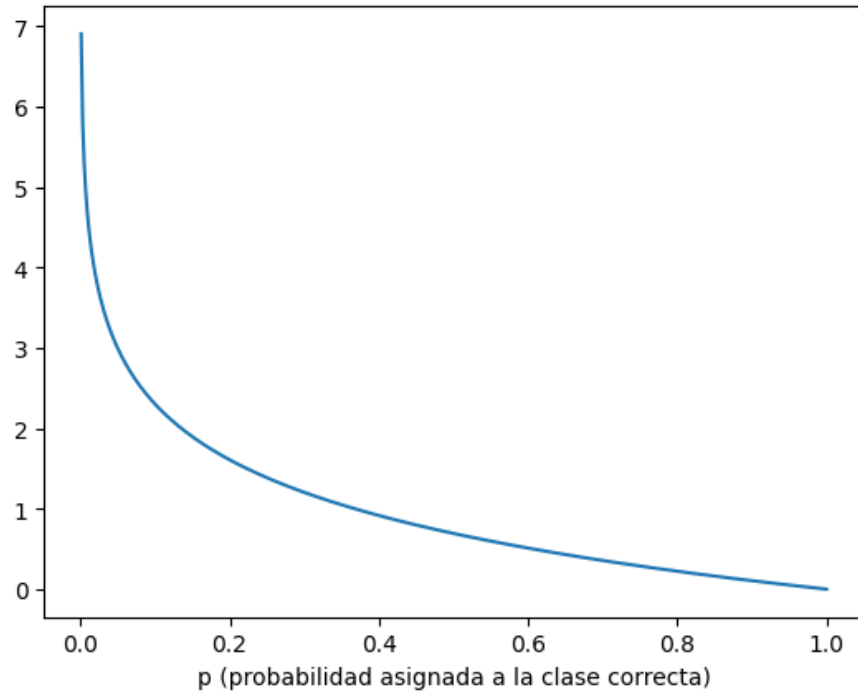
[0,0,0,0,0,0,0,0,0,1]

El "Hello word" del DL

Step-by-step

$$CE = - \sum_{i=1}^N y_i \log(p_i) \quad \text{softmax}(e^{z_i}) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

Supongamos → Etiqueta verdadera: $y = 6$



TODO se anula excepto: $CE = -\log(0.771)$
 $CE = 0.26$

Supongamos → Probabilidad de 0.05

$CE = -\log(0.05)$
 $CE = 3$

El "Hello word" del DL

Step-by-step

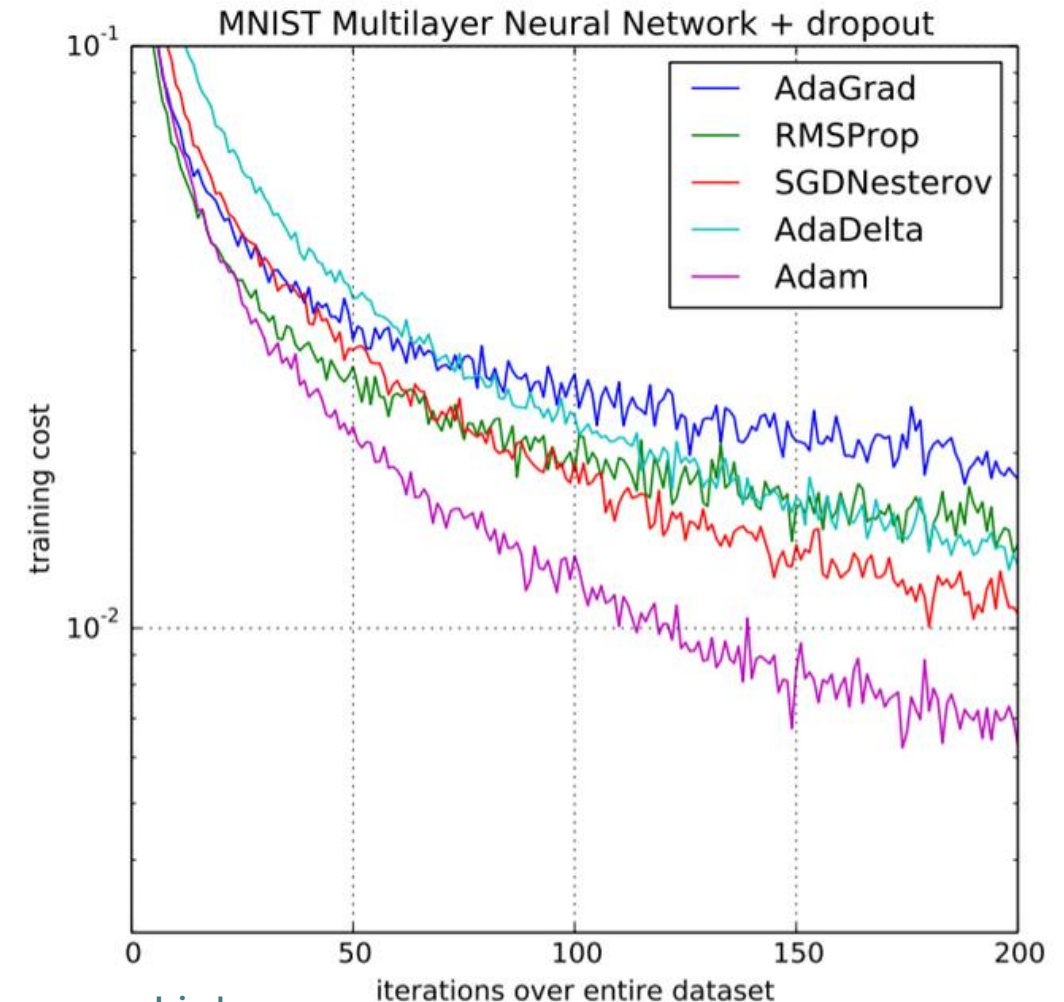
```
optimizer = Adam(model.parameters())
```

Seleccione el optimizador que actualiza los parámetros del modelo durante el entrenamiento

Optimizer: ¿quién actualiza los pesos?

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(),
                               lr=1e-3)
```

Ajusta el learning rates durante el training.



El "Hello word" del DL

Step-by-step

```
def get_batch_accuracy(output, y, N):  
    pred = output.argmax(dim=1, keepdim=True)  
    correct = pred.eq(y.view_as(pred)).sum().item()  
    return correct / N
```

- argmax selecciona la clase con mayor score
- Comparar pred vs y produce True/False
- Accuracy = correctas / total

Función para calcular la precisión (accuracy)

	Class 0	Class 1	Class 2	
Image 1	0.2	2.8	0.5	argmax { 2.8 Class 1 3.1 Class 0 2.5 Class 2 2.1 Class 1
Image 2	3.1	0.4	0.2	
Image 3	0.1	0.8	2.5	
Image 4	0.5	2.1	1.2	

	pred			
Image	Prediction	True label y	Correct	
1	1	1	✓	
2	0	0	✓	
3	2	2	✓	
4	1	0	✗	

$$\text{correct} / N = \frac{3}{4} = 0.75 \rightarrow 75\%$$

accuracy

El "Hello word" del DL

Step-by-step

```
def get_batch_accuracy(output, y, N):  
    pred = output.argmax(dim=1, keepdim=True)  
    correct = pred.eq(y.view_as(pred)).sum().item()  
    return correct / N
```

**Función para
calcular la precisión
(accuracy)**

Supongamos que la red procesa 5 imágenes

OUTPUT: Salidas de la red neuronal (logits o probabilidades)

Forma: (5, 10) -> 5 imágenes, 10 posibles dígitos

```
output = torch.tensor([  
    [0.1, 0.05, 0.05, 0.7, 0.02, 0.03, 0.01, 0.02, 0.01, 0.01], # Imagen 1  
    [0.9, 0.02, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01], # Imagen 2  
    [0.05, 0.8, 0.05, 0.02, 0.02, 0.01, 0.01, 0.01, 0.01, 0.02], # Imagen 3  
    [0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.85, 0.05, 0.04], # Imagen 4  
    [0.02, 0.01, 0.75, 0.05, 0.05, 0.03, 0.03, 0.02, 0.02, 0.02] # Imagen 5  
)
```

Y: Etiquetas verdaderas (qué dígito es realmente cada imagen)

```
y = torch.tensor([3, 0, 1, 7, 2])
```

El "Hello word" del DL

Step-by-step

```
pred = output.argmax(dim=1, keepdim=True)
```

```
output = torch.tensor([
    [0.1, 0.05, 0.05, 0.7, 0.02, 0.03, 0.01, 0.02, 0.01, 0.01], # Imagen 1 → 3
    [0.9, 0.02, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01], # Imagen 2 → 0
    [0.05, 0.8, 0.05, 0.02, 0.02, 0.01, 0.01, 0.01, 0.01, 0.02], # Imagen 3 → 1
    [0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.85, 0.05, 0.04], # Imagen 4 → 7
    [0.02, 0.01, 0.75, 0.05, 0.05, 0.03, 0.03, 0.02, 0.02, 0.02] # Imagen 5 → 2
])
```

```
pred = [[3], [0], [1], [7], [2]]
```

```
correct = pred.eq(y.view_as(pred)).sum().item()
```

```
pred.eq(y.view_as(pred)) = [[True], # 3 == 3 ✓
                             [True], # 0 == 0 ✓
                             [True], # 1 == 1 ✓
                             [True], # 7 == 7 ✓
                             [True]] # 2 == 2 ✓
```

```
return correct / N # return 5 / 5 = 1.0 (100% Accuracy)
```

```
.sum().item() = 5 # ¡Todas correctas!
```

Step-by-step

```
epochs = 15

for epoch in range(epochs):
    print('Epoch: {}'.format(epoch))
    train()
    validate()
```

Train = aprender

Validate = examinar

**Entrenar el
modelo**

```
def train():
    loss = 0
    accuracy = 0

    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        output = model(x)
        optimizer.zero_grad()
        batch_loss = loss_function(output, y)
        batch_loss.backward()
        optimizer.step()

        loss += batch_loss.item()
        accuracy += get_batch_accuracy(output, y, train_N)
    print('Train - Loss: {:.4f} Accuracy: {:.4f}'.format(loss, accuracy))
```

```
def validate():
    loss = 0
    accuracy = 0

    model.eval()
    with torch.no_grad():
        for x, y in valid_loader:
            x, y = x.to(device), y.to(device)
            output = model(x)

            loss += loss_function(output, y).item()
            accuracy += get_batch_accuracy(output, y, valid_N)
    print('Valid - Loss: {:.4f} Accuracy: {:.4f}'.format(loss, accuracy))
```

El "Hello word" del DL

Step-by-step

Epoch: 0 Train - Loss: 385.4550 Accuracy: 0.9378
Epoch: 1 Train - Loss: 161.4343 Accuracy: 0.9733
Epoch: 2 Train - Loss: 108.4484 Accuracy: 0.9808
Epoch: 3 Train - Loss: 85.1808 Accuracy: 0.9852
Epoch: 4 Train - Loss: 64.7815 Accuracy: 0.9892
Epoch: 5 Train - Loss: 54.5347 Accuracy: 0.9907
Epoch: 6 Train - Loss: 48.0643 Accuracy: 0.9922
Epoch: 7 Train - Loss: 42.1750 Accuracy: 0.9926
Epoch: 8 Train - Loss: 36.8053 Accuracy: 0.9938
Epoch: 9 Train - Loss: 33.0069 Accuracy: 0.9946
Epoch: 10 Train - Loss: 36.2901 Accuracy: 0.9938
Epoch: 11 Train - Loss: 29.2552 Accuracy: 0.9956
Epoch: 12 Train - Loss: 25.9809 Accuracy: 0.9959
Epoch: 13 Train - Loss: 29.9886 Accuracy: 0.9953
Epoch: 14 Train - Loss: 30.6800 Accuracy: 0.9957

Valid - Loss: 32.4810 Accuracy: 0.9672
Valid - Loss: 27.6381 Accuracy: 0.9744
Valid - Loss: 27.6959 Accuracy: 0.9767
Valid - Loss: 38.8592 Accuracy: 0.9664
Valid - Loss: 28.3647 Accuracy: 0.9755
Valid - Loss: 26.6371 Accuracy: 0.9791
Valid - Loss: 23.7243 Accuracy: 0.9804
Valid - Loss: 28.8064 Accuracy: 0.9782
Valid - Loss: 29.6902 Accuracy: 0.9792
Valid - Loss: 30.3106 Accuracy: 0.9825
Valid - Loss: 35.1231 Accuracy: 0.9800
Valid - Loss: 28.1439 Accuracy: 0.9831
Valid - Loss: 41.6936 Accuracy: 0.9774
Valid - Loss: 38.3812 Accuracy: 0.9783
Valid - Loss: 36.6485 Accuracy: 0.9820

Decidir **cuál es la mejor época**, normalmente **NO se hace con las métricas de Train**.

Se hace con las métricas de **Validation**, se desea es seleccionar el modelo que mejor generaliza a datos que no está usando para actualizar los pesos.

Early Stopping + Checkpoints

Checkpoint

- Guarda el mejor estado del modelo
- Permite recuperar pesos después
- Se activa cuando mejora la métrica

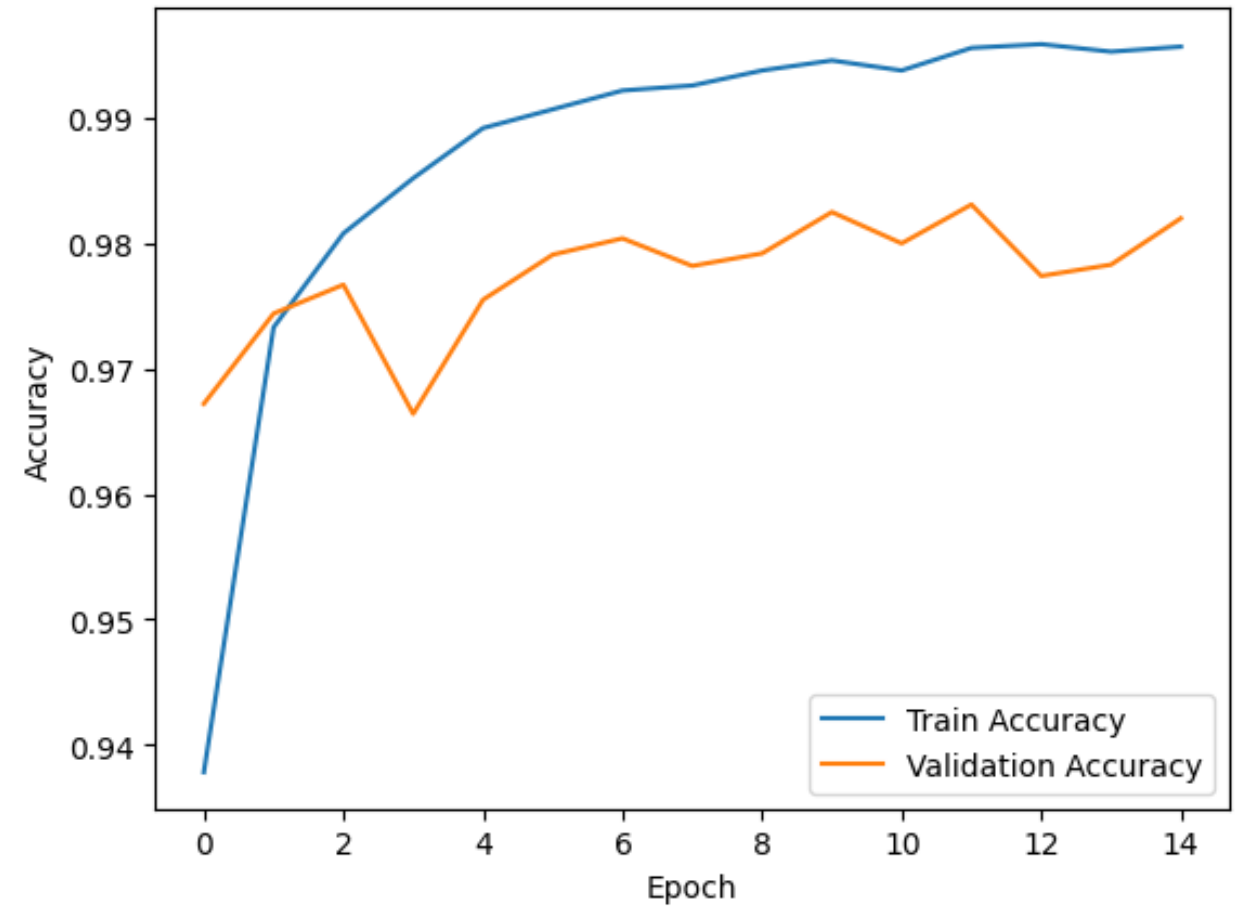
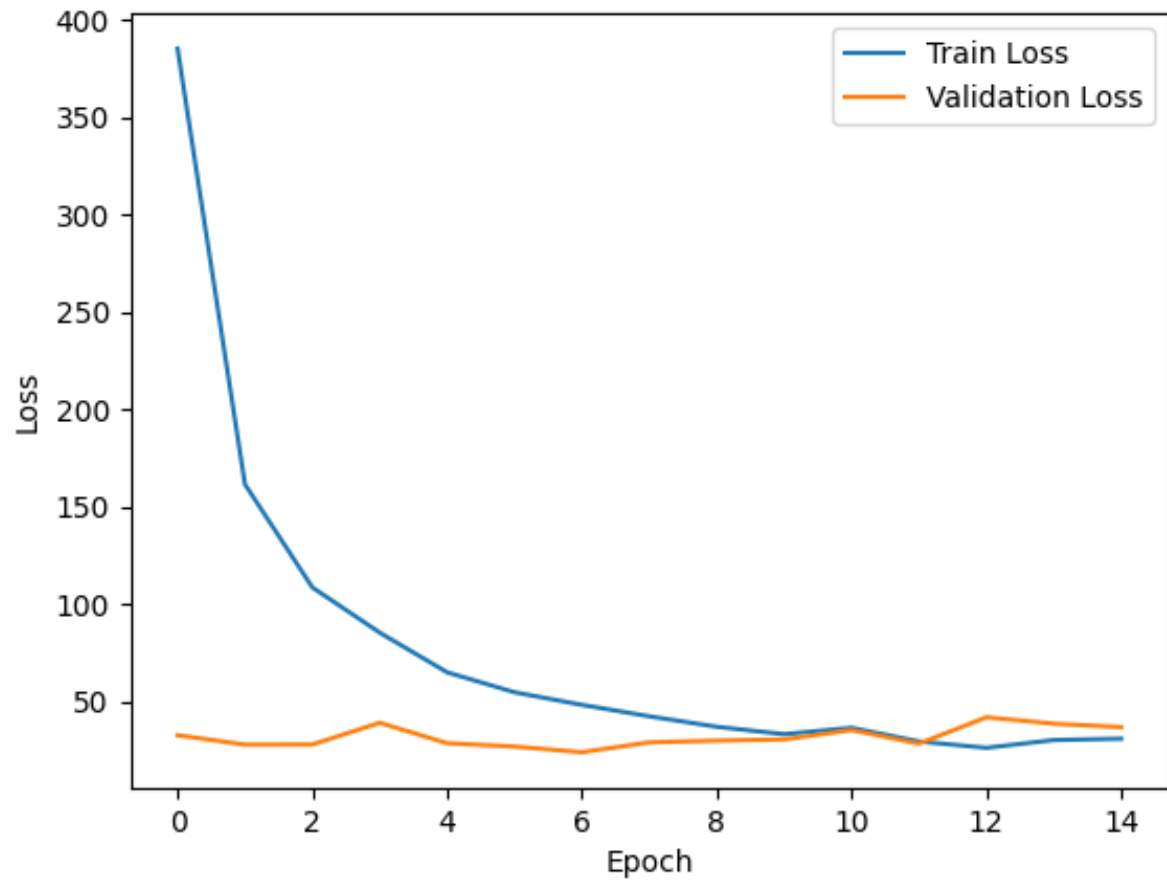
Early stopping

- Detiene cuando validation deja de mejorar
- Usa patience
- Evita entrenamiento innecesario

```
Epoch: 3
Train - Loss: 83.0794 Accuracy: 0.9860
Valid - Loss: 24.1464 Accuracy: 0.9785
Epoch: 4
Train - Loss: 67.2245 Accuracy: 0.9888
Valid - Loss: 30.4662 Accuracy: 0.9752
Epoch: 5
Train - Loss: 53.1826 Accuracy: 0.9908
Valid - Loss: 21.8572 Accuracy: 0.9816
Epoch: 6
Train - Loss: 49.8736 Accuracy: 0.9916
Valid - Loss: 23.8648 Accuracy: 0.9822
Epoch: 7
Train - Loss: 41.9180 Accuracy: 0.9926
Valid - Loss: 26.2729 Accuracy: 0.9820
Epoch: 8
Train - Loss: 36.4449 Accuracy: 0.9940
Valid - Loss: 28.5248 Accuracy: 0.9806
Epoch: 9
Train - Loss: 35.7901 Accuracy: 0.9940
Valid - Loss: 29.1322 Accuracy: 0.9815
Early stopping triggered after epoch 9 (no improvement for 4 epochs).
Restored best model (validation loss = 21.8572).
```

Checkpoint guarda el mejor modelo · Early stopping decide cuándo parar

El "Hello word" del DL



El "Hello word" del DL

```
tensor([[ -56.3176, -47.4944, -58.1196,  2.6680, -54.7834, 32.3323, -29.6542, -40.3872, -37.2434, -25.3665]],
```

```
prediction.argmax(dim=1, keepdim=True) tensor([[5]])
```

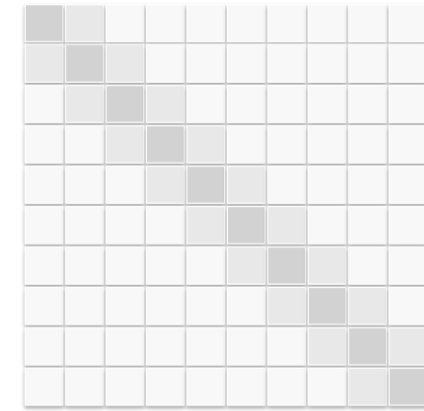
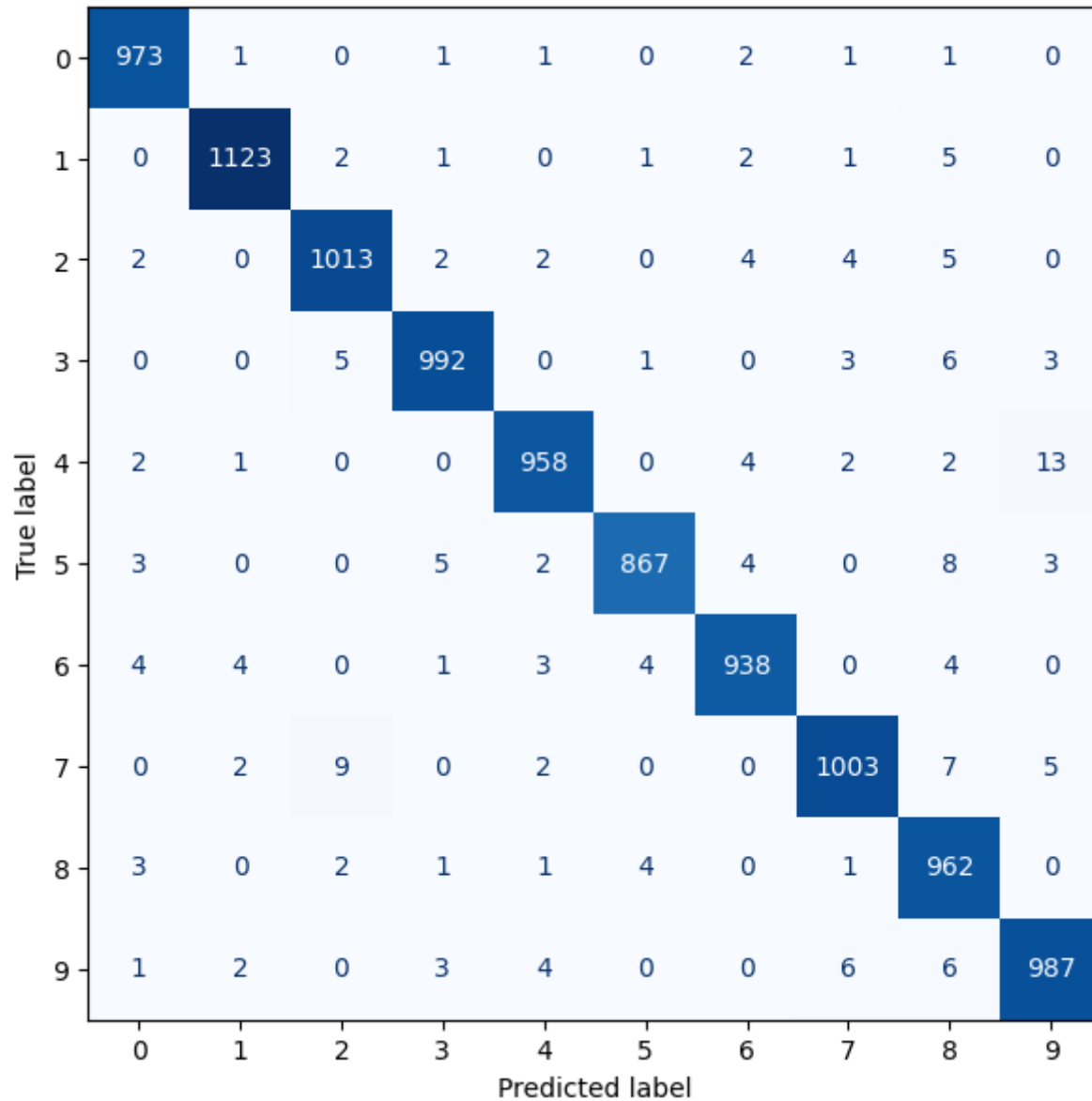
`y_0` 5

T:7	T:2	T:1	T:0	T:4	T:1	T:4	T:9	T:5	T:9	T:0	T:6	T:9	T:0	T:1	T:5	T:9	T:7	T:3	T:4	T:9	T:6	T:6	T:5	T:4
P:7	P:2	P:1	P:0	P:4	P:1	P:4	P:9	P:6	P:9	P:0	P:6	P:9	P:0	P:1	P:5	P:9	P:7	P:3	P:4	P:9	P:6	P:6	P:5	P:4
7	2	1	0	4	1	4	9	5	9	0	6	9	0	1	5	9	7	3	4	9	6	6	5	4

¿Cómo guardar el modelo e implementarlo para futuras predicciones ?

El "Hello word" del DL

Confusion Matrix — Validation Set

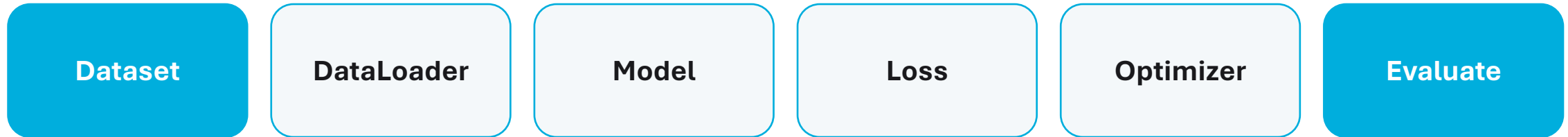


- Diagonal fuerte = predicciones correctas
- Fuera de diagonal = pares de clases confundidas
- Busca patrones como $4 \leftrightarrow 9$, $3 \leftrightarrow 5$ o $7 \leftrightarrow 1$

Quiz

- ¿Cuál es la diferencia entre Dataset y DataLoader?
- ¿Por qué shuffle=True en entrenamiento y normalmente False en validación?
- ¿Qué hace nn.Flatten() y por qué conserva la dimensión batch?
- ¿Por qué NO se debe aplicar Softmax antes de CrossEntropyLoss?
- ¿Qué diferencia hay entre model.eval() y torch.no_grad()?
- ¿Qué problema indica train_accuracy↑ mientras valid_accuracy se estanca o baja?
- ¿Qué garantiza un checkpoint que el entrenamiento normal no garantiza?

Key Takeaways



- Un pipeline DL completo no es solo “definir una red”.
- Train/validation permite detectar problemas de generalización.
- device, eval/no_grad, checkpoints y curvas hacen el código más robusto.
- MNIST es el punto de partida; el siguiente paso natural son las CNNs.

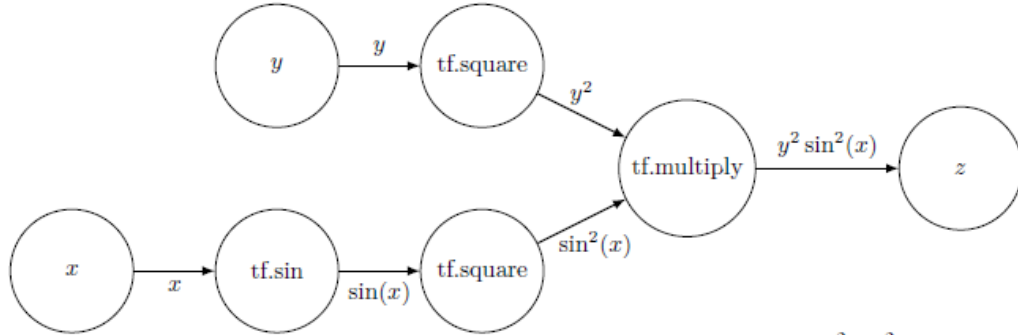
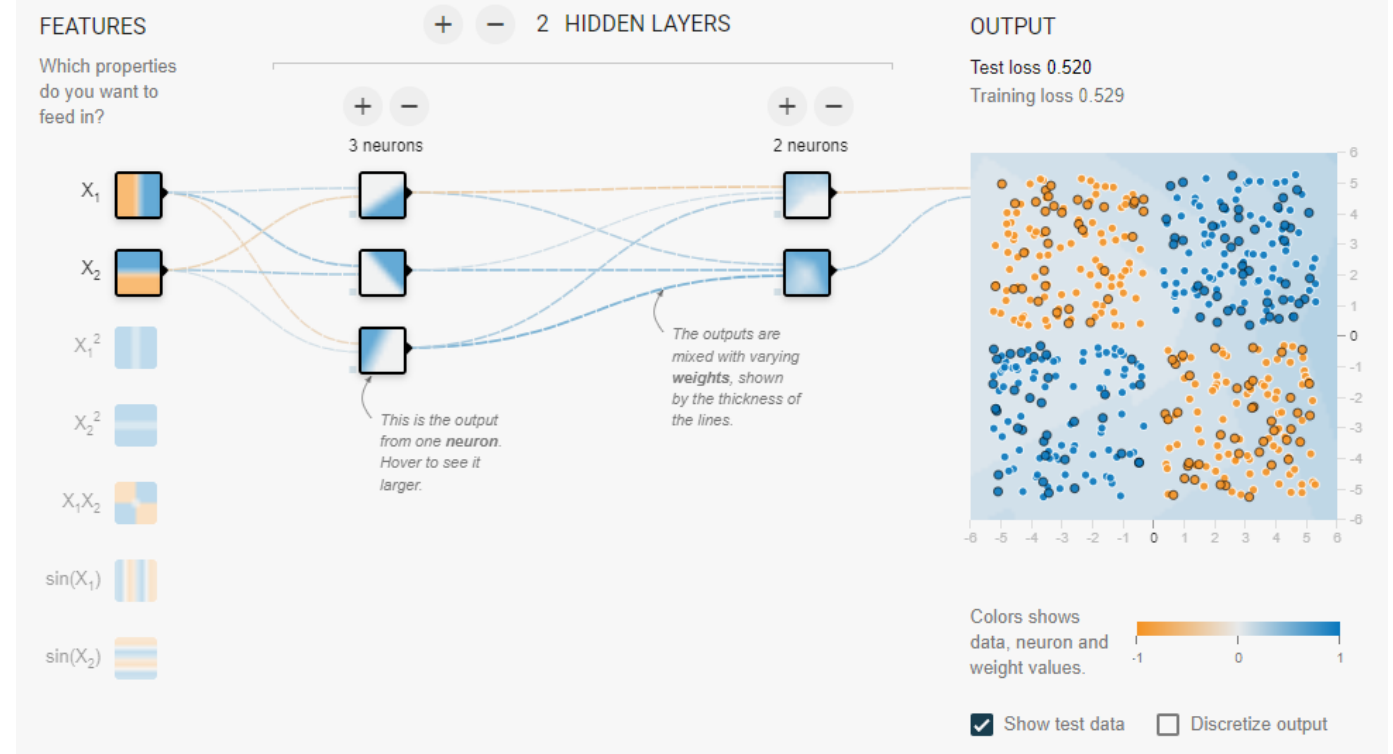


Figure 2.1: A directed graph representation of computing $z = y^2 \sin^2(x)$



[A Neural Network Playground \(tensorflow.org\)](https://tfplay.tensorflow.org/)